

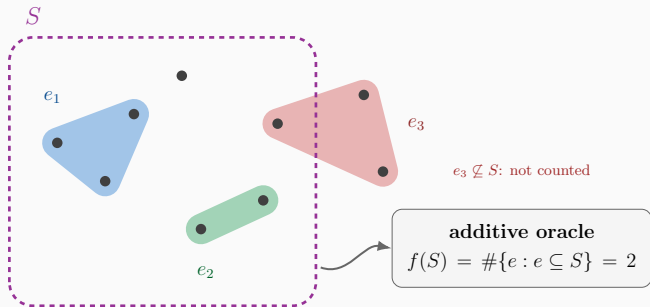
Adaptive Sparse Möbius Transforms for Learning Polynomials

Yigit Efe Erginbas Justin Kang Elizabeth Polito Kannan Ramchandran

University of California, Berkeley

Introduction

Motivation: reconstructing hidden combinatorial structure



- Hidden hypergraph: circuits, metabolic networks, communities, ...
- We may ask **additive queries**: how many hyperedges inside S ?
- Goal: **exact** reconstruction with as few queries as possible.

From hypergraphs to sparse polynomials

Encode the queried vertex subset as $\mathbf{x} \in \{0, 1\}^n$. The additive oracle is a **polynomial**:

$$f(\mathbf{x}) = \sum_{\mathbf{k} \leq \mathbf{x}} F(\mathbf{k}) \quad \text{— sum of weights } F(\mathbf{k}) \text{ of hyperedges } \mathbf{k} \text{ inside the queried set}$$

- s hyperedges of size $\leq d \Leftrightarrow s$ -**sparse** polynomial of **degree** d .

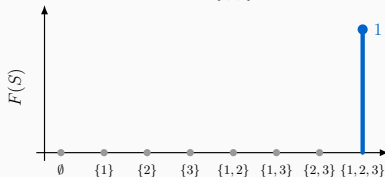
Goal: sparse Möbius transform

Exactly recover all pairs $(\mathbf{k}, F(\mathbf{k}))$ from few evaluation queries $f(\mathbf{x})$.

Why not Fourier analysis?

AND basis (Möbius)

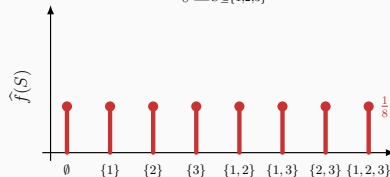
$$f = \phi_{\{1,2,3\}}$$



1 nonzero coefficient

Parity basis (Fourier)

$$f = \frac{1}{8} \sum_{S \subseteq \{1,2,3\}} \chi_S$$

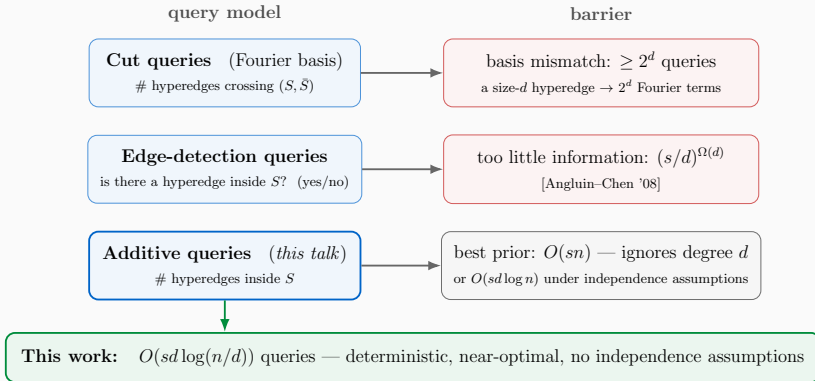


2^d nonzero coefficients

Conjunctions are **local**; parities are **global**.

Fourier-based methods (cut queries) must pay 2^d for a single size- d hyperedge.

Prior approaches: every query model hits a barrier



FASMT (fully adaptive)

$O(sd \log(n/d))$ queries, polynomial time

near-optimal

PASMT (partially adaptive)

$O(sd^2 \log n)$ queries in only $O(d^2 \log n)$ rounds

rounds independent of s

Lower bound

$\Omega(sd \log(n/d) / \log s)$ queries required by *any* algorithm

Deterministic, constructive, no independence assumptions.

Queries are Group Tests

Evaluations are group tests over the Boolean semiring

$$f(\mathbf{x}) = \sum_{\mathbf{k} \leq \mathbf{x}} F(\mathbf{k})$$

evaluation = subset sum

$$\mathbf{k} \leq \mathbf{x} \iff \mathbf{h}^\top \mathbf{k} = 0, \quad \mathbf{h} := \neg \mathbf{x}$$

group test on the support of F

$$f(\mathbf{1}) - f(\neg \mathbf{h}) = \sum_{\mathbf{h}^\top \mathbf{k} = 1} F(\mathbf{k}), \quad f(\neg \mathbf{h}) = \sum_{\mathbf{h}^\top \mathbf{k} = 0} F(\mathbf{k})$$

one query splits the space

Key insight

Group testing designs act as *projections of the coefficient space*:

we can navigate the subset lattice by iteratively splitting with test vectors $\mathbf{h}$.

Divide and conquer on the coefficient space

Split coefficients into **bins** by test outcomes: starting from $\mathcal{B}(\epsilon) = \{0, 1\}^n$,

$$\mathcal{B}(\ell \| 0) = \{\mathbf{k} \in \mathcal{B}(\ell) : \mathbf{h}^\top \mathbf{k} = 0\}, \quad \mathcal{B}(\ell \| 1) = \{\mathbf{k} \in \mathcal{B}(\ell) : \mathbf{h}^\top \mathbf{k} = 1\}$$

with **bin sums** $V(\ell) = \sum_{\mathbf{k} \in \mathcal{B}(\ell)} F(\mathbf{k}) = V(\ell \| 0) + V(\ell \| 1)$.

- **Prune:** $V(\ell) = 0 \Rightarrow$ discard $\mathcal{B}(\ell)$.¹
- **Peel:** single coefficient left $\Rightarrow$ value = bin sum, location = outcomes ℓ .
- Tree has width $\leq s$ (sparsity) — pruning keeps the search small.

¹Safe under a mild non-degeneracy assumption (e.g., satisfied almost surely by random weights).

The catch: queries mix bins

No query isolates a single bin. The best we can do (query vector $\mathbf{x}_\ell$):

$$f(\mathbf{x}_\ell) = \sum_{\mathbf{H}^\top \mathbf{k} \leq \ell} F(\mathbf{k}) \quad \text{— the target bin plus all bins below it}$$

PASMT: shared tests

Fixed matrix $\mathbf{H}$ for all bins

$\Rightarrow$ structured linear system,
solve all bin sums per round

FASMT: adaptive tests

Per-bin test vectors

$\Rightarrow$ process bins in order,
subtract already-found coefficients

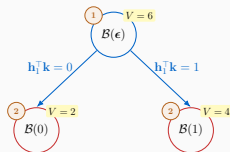
Two Algorithms

PASMT: breadth-first splitting with a fixed design



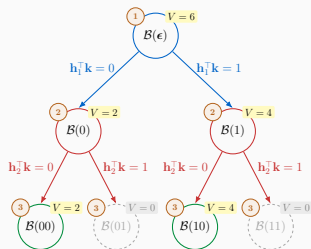
All bins at depth t split with the **same** column $\mathbf{h}_t$ of a d -disjunct matrix $\mathbf{H}$.

PASMT: breadth-first splitting with a fixed design



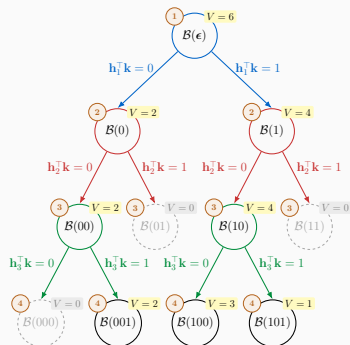
All bins at depth t split with the **same** column $\mathbf{h}_t$ of a d -disjunct matrix $\mathbf{H}$.

PASMT: breadth-first splitting with a fixed design



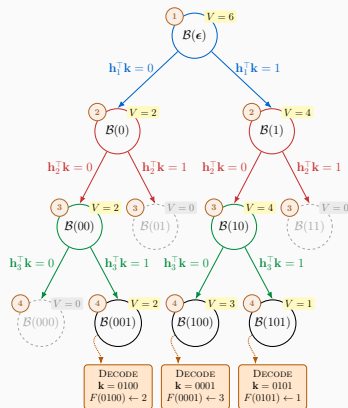
All bins at depth t split with the **same** column h_t of a d -disjunct matrix $\mathbf{H}$.

PASMT: breadth-first splitting with a fixed design



All bins at depth t split with the **same** column h_t of a d -disjunct matrix $\mathbf{H}$.

PASMT: breadth-first splitting with a fixed design



All bins at depth t split with the **same** column h_t of a d -disjunct matrix H .

PASMT: why it works

- **H**: *d*-disjunct group testing matrix with $b = O(d^2 \log n)$ columns.
- Each round: queries form a *lower-triangular* linear system in the bin sums
 $\Rightarrow$ back-substitution recovers every active bin, one query per bin.
- After b rounds each surviving bin holds one coefficient;
location decoded from its label: $\ell = \mathbf{H}^\top \mathbf{k} \Rightarrow \mathbf{k} = \text{DECODE}(\mathbf{H}, \ell)$.

Theorem (PASMT)

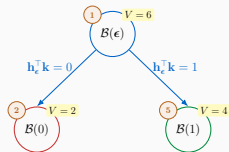
Exact recovery with $O(sd^2 \log n)$ queries in $O(d^2 \log n)$ rounds — independent of s .

FASMT: depth-first search with adaptive tests



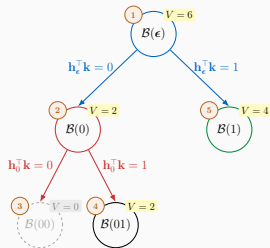
Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



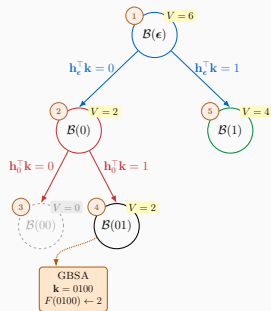
Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



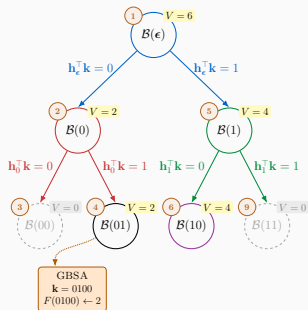
Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



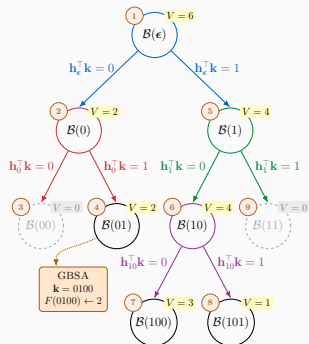
Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



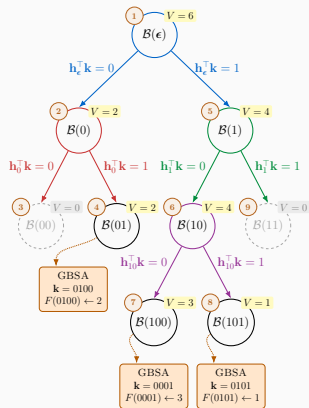
Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: depth-first search with adaptive tests



Test vectors chosen **adaptively** per bin by generalized binary splitting (GBSA).

FASMT: why it works

- GBSA [Hwang '72] pins down a $\leq d$ -sparse $\mathbf{k}$ in $O(d \log(n/d))$ tests
 $\Rightarrow$ only $O(d \log(n/d))$ splits per coefficient (vs. $O(d^2 \log n)$ fixed tests).
- Adaptive tests break the linear-system structure.

Fix: process bins **depth-first** and query the **residual**

$$f_T(\mathbf{x}) = f(\mathbf{x}) - \sum_{\mathbf{k} \leq \mathbf{x}} T(\mathbf{k}), \quad T = \text{coefficients found so far,}$$

so each split still costs a **single** query.

Theorem (FASMT)

Exact recovery with $O(sd \log(n/d))$ queries, in polynomial time.

Two ends of an adaptivity trade-off

	PASMT	FASMT
traversal	breadth-first	depth-first
test vectors	fixed d -disjunct $\mathbf{H}$	adaptive (GBSA)
queries	$O(sd^2 \log n)$	$O(sd \log(n/d))$
adaptive rounds	$O(d^2 \log n)$	$O(sd \log(n/d))$

Choose by bottleneck: total measurements (FASMT) vs. parallelism (PASMT).

Lower Bound

FASMT is near-optimal

Counting argument:

- Many candidates: describing f takes $\Omega(sd \log(n/d))$ bits.
- Little information per query: each answer reveals only $O(\log s)$ bits.

Theorem (lower bound)

Any deterministic algorithm needs $\Omega\left(\frac{sd \log(n/d)}{\log s}\right)$ evaluation queries.

FASMT's $O(sd \log(n/d))$ matches up to $O(\log s)$ — in every scaling regime of s, d .

Conclusion

Takeaways & open questions

Takeaways

- Möbius (AND) basis keeps conjunctive structure sparse — and group testing turns evaluation queries into a constructive search over the coefficient lattice.
- FASMT: near-optimal $O(sd \log(n/d))$; PASMT: few rounds, independent of s .

Open questions

- Noisy oracles — import robust group testing?
- Fewer adaptive rounds at $O(sd \log n)$ queries?
- Is the non-cancellation assumption necessary?

Thank you!